

Sistemas en Tiempo Real

3º Curso – Grado en Ingeniería Informática
Especialidad Ingeniería de Computadores



Sistemas en Tiempo Real

Programa de Teoría

Tema 1: Introducción a los Sistemas en Tiempo Real (2 h.)

Tema 2: Lenguajes para aplicaciones en Tiempo Real (2 h.)

Tema 3: Interfaces y Elementos Hardware (4 h.)

Tema 4: Concurrencia y Sincronización (6 h.)

Tema 5: Sistemas Operativos en Tiempo Real (6 h.)

Tema 6: Planificación de Tiempo Real (6 h.)



Tema 4. Concurrencia y Sincronización



1.Programación con memoria compartida

- 1.Conceptos básicos de programas concurrentes
- 2.Paradigmas de programación concurrente
- 3.Ejecución atómica de instrucciones
- 4.Seguridad y Viveza

2.Secciones Críticas: Garantía de Exclusión Mutua

3.Cerrojos, Barreras y Semáforos

4.Monitores

5.Programación por Paso de Mensajes



Tema 4. Concurrencia y Sincronización

- Conceptos básicos de programas concurrentes
 - Dos o más procesos que se ejecutan simultáneamente trabajando de manera conjunta para realizar una tarea.
 - Tareas:
 - Independientes.
 - Cooperativas.
 - Competitivas.
 - Su comportamiento puede variar a lo largo del tiempo.
 - Necesidad de que las tareas se ejecuten en paralelo siempre que sea posible y de manera ordenada cuando necesiten coordinarse. Comunicación y sincronización:
 - Variables compartidas.
 - Paso de mensajes.



Tema 4. Concurrencia y Sincronización

● Paradigmas de programación concurrente:

- Paralelismo iterativo
 - Cada *for* se realiza como una operación concurrente (se acceden a datos independientes).
- Paralelismo recursivo
 - Cada función se ejecuta como una tarea concurrente.
- Productor/Consumidor
 - Existen tareas que producen datos y otras tareas que procesan esos datos de manera concurrente. Comunicación en un solo sentido.
- Cliente/Servidor
 - Existen procesos servidores que atienden las peticiones de otros procesos clientes y proporcionan respuestas a los mismos. Comunicación bidireccional.
- Conjunto de Compañeros (*Peers*)
 - Existen procesos independientes que se encargan de realizar una tarea de manera concurrente que se comunican intercambiando datos según los van necesitando.



Tema 4. Concurrency and Synchronization

- **Parallelization of algorithms: Independence of the parallelized parts**
 - Dos partes de un algoritmo son independientes: si la intersección de sus conjuntos de escritura es el conjunto vacío y si la intersección entre el conjunto de escritura de cada uno de ellos con el conjunto de lectura del otro (en cada caso) es el conjunto vacío.
 - Se define como conjunto de lectura de un proceso a todas aquellas variables que dicho proceso lee (sin modificar) a lo largo de su ejecución.
 - Se define como conjunto de escritura de un proceso a todas aquellas variables que son creadas o modificadas por dicho proceso a lo largo de su ejecución.
 - Se produce independencia si ambas partes:
 - Solo acceden a variables locales.
 - Si solo "leen" variables compartidas.
 - Si "escriben" en diferentes variables compartidas en cada caso.
 - Cuando dos partes paralelas no son totalmente independientes hay que arbitrar mecanismos para conseguir que la escritura y lectura de las variables compartidas no accedan simultáneamente.



Tema 4. Concurrencia y Sincronización

• Ejecución Atómica de Instrucciones

- **Ejecución atómica:** ejecución como una acción **única indivisible**.
- **En ensamblador,** las instrucciones pueden durar varios ciclos de reloj, sin embargo, está garantizado que la ejecución de la instrucción completa se realizará sin interrupciones (**ejecución atómica**).
- Al garantizar que una instrucción no es interrumpida hasta que se ejecuta completamente, una instrucción de otro proceso no comenzará a ejecutarse en mitad de la ejecución de otra.
- Sin embargo, esto no es cierto para instrucciones de alto nivel, ya que pueden estar compuestas por varias instrucciones en ensamblador.



Tema 4. Concurrencia y Sincronización

- **Atomicidad o Paralelismo de Grano Fino**
 - La ejecución atómica de las instrucciones está garantizada mediante **mecanismos hardware.**
- **Atomicidad o Paralelismo de Grano Grueso**
 - La ejecución atómica de las instrucciones se garantiza mediante **mecanismos software.**
 - Paralelismo de **Grano Medio**
 - Hace referencia a **bloques de código paralelo** dentro de una misma tarea. **Alto nivel de cooperación/competición => Mucha sincronización.**
 - Paralelismo de **Grano Grueso**
 - Hace referencia a la ejecución en paralelo de diversas tareas dentro de un mismo sistema. **Bajo nivel de cooperación/competición => Poca sincronización.**



Tema 4. Concurrencia y Sincronización

Paralelismo iterativo

• Cálculo del máximo en • Bucle en ensamblador: un array:

```
#define TAM n
int m = 0; // Compartida Escritura
int a[TAM]; // Compartida Lectura
int i; // Privada

omp_set_num_threads(TAM);
#pragma omp parallel shared(m,a)
private(i)
{
    #pragma omp for schedule(dynamic)
    nowait
    {
        for (i=0; i < TAM; i++){
            if (a[i] > m)
                m = a[i];
        }
    }
}
```

```
MOV SI,0 ; (i = 0)
BUCLE_FOR: MOV AX, A[SI] ; a[i]
            CMP AX, [m] ; (a[i] > m)
            JBE CONTINUA
            MOV [m], AX ; m = a[i]
CONTINUA: INC SI ; i++
            ; i < TAM => Bucle for
            CMP SI, TAM
            JB BUCLE_FOR
```

Si por casualidad no tengo tanto núcleos de procesamiento para cumplir el número de hilos, cuando acaba uno, sigue cogiendo sin orden hilos

Esto no funciona, ya que no es atómico, con lo que se puede modificar dos a la vez y producir error, machamiento de datos ya que la var m es compartida en Escritura



Tema 4. Concurrencia y Sincronización

● Cálculo del máximo en un array: ● Bucle en ensamblador:

```

#define TAM n
int m = 0; // Compartida Escritura
int a[TAM]; // Compartida Lectura
int i; // Privada

omp_set_num_threads(TAM);
#pragma omp parallel shared(m,a) private(i)
{
    #pragma omp for schedule(dynamic)
    nowait
    {
        for (i=0; i < TAM; i++){
            EJECUCION ATOMICA {
                if (a[i] > m)
                    m = a[i];
            }
        }
    }
}

```

```

MOV SI, 0 ; (i = 0)
BUCLE_FOR: CLI ; BLOQUEO INTERRUPCIONES
MOV AX, A[SI] ; a[i]
CMP AX, [m] ; (a[i] > m)
JBE CONTINUA
MOV [m], AX ; m = a[i]
CONTINUA: STI ; HABILITAR INTERRUP.
INC SI ; i++
; i < TAM => Bucle for
CMP SI, TAM
JB BUCLE_FOR

```

Evitamos los problemas anteriores inhabilitando las interrupciones y cuando se acabe el bloque de ejecución atómica se habilitan de nuevo

NO ES ÚTIL, ya que si el bucle es atómico es igual que si lo ejecutamos secuencial, incluso tarda mas ya que tienes que crear los hilos y capturarlos.

Al ser dinámico y no seguir orden da muchos fallos de caché, el paralelismo no siempre es útil.

Tema 4. Concurrencia y Sincronización

• Cálculo del máximo en un array:

```
#define TAM n
int m = 0; // Compartida Escritura
int a[TAM]; // Compartida Lectura
int i; // Privada

omp_set_num_threads(TAM);
#pragma omp parallel shared(m,a)
private(i)
{
    #pragma omp for schedule(dynamic)
    nowait
    {
        for (i=0; i < TAM; i++){
            EJECUCION_ATOMICA {
                if (a[i] > m)
                    m = a[i];
            }
        }
    }
}
```

- Al ejecutarse de manera atómica todo el cuerpo del bucle es equivalente a ejecutarla secuencialmente.
- En el secuencial, todos los elementos de la matriz son evaluados desde 0 hasta TAM-1; mientras que en el caso concurrente, el orden de evaluación es arbitrario.



Tema 4. Concurrencia y Sincronización

• Cálculo del máximo en un array:

```
#define TAM      n
int m = 0;  // Compartida Escritura
int a[TAM]; // Compartida Lectura
int i; // Privada

omp_set_num_threads(TAM);
#pragma omp parallel shared(m,a)
private(i)
{
    #pragma omp for schedule(dynamic)
    nowait
    {
        for (i=0; i < TAM; i++){
            if (a[i] > m)
                EJECUCION ATOMICA {
                    if (a[i] > m)
                        m = a[i];
                }
        }
    }
}
```



Para decir que es
modificada externa
(en c/c++ extern)

• En este caso lo más eficiente es realizar una *doble evaluación*.

• La 1ª evaluación se puede hacer en paralelo, puesto que es de solo lectura. Mediante ésta se conseguirá “eliminar” a una gran parte de las tareas paralelas (porque serán menores que m)
=> No ejecutarán el cuerpo del bucle.

• La 2ª evaluación tiene que realizarse junto con la asignación de manera atómica para garantizar que entre medias no interrumpe otra tarea.



Tema 4. Concurrency and Synchronization

- Interferencia entre tareas:
 - Un conjunto de tareas se consideran libres de interferencias si ninguna asignación en ninguna tarea impide el cumplimiento de las afirmaciones críticas (postcondiciones) de otras tareas.
- Técnicas para evitar la interferencia entre tareas:
 - Variables disjuntas
 - Los conjuntos de escritura de las tareas no se solapan.
 - Afirmaciones debilitadas
 - Cuando hay solapamientos entre los conjuntos de escritura y lectura de las tareas, se pueden asumir afirmaciones (postcondiciones) menos estrictas que tengan en cuenta la ejecución concurrente.
 - Invariantes globales
 - Emplear invariantes globales que capturen las relaciones entre las variables globales en todo momento.
 - Sincronización
 - Utilizar mecanismos que ordenen la ejecución de los bloques de código para impedir su ejecución concurrente.



Tema 4. Concurrencia y Sincronización

• Conjuntos disjuntos

• Calcular: $B[i] = A[i-1]*-1 + A[i]*2 + A[i+1]*-1;$

```
int A[TAM], b1[TAM], b2[TAM], b3[TAM];
int B[TAM];
int i;

for (i=0; i > TAM/2; i++)
    b1[i]=b2[i]=0;

#pragma omp parallel shared(A,b1,b2,b3)
    private(i)
    {
        #pragma omp sections
        {
            #pragma omp section
            {
                for(i=0; i<TAM-1; i++)
                    b1[i+1] = A[i]*(-1);
            }

```

```
        #pragma omp section
        {
            for(i=0; i<TAM; i++)
                b2[i] = A[i]*2;
        }
        #pragma omp section
        {
            for(i=1; i<TAM; i++)
                b3[i-1] = A[i]*(-1);
        }
    }

for (i=0; i<TAM; i++)
    B[i] = b1[i]+b2[i]+b3[i];

```



Tema 4. Concurrency y Sincronización

• Afirmaciones debilitadas

```
int x = 0;
```

```
#pragma omp parallel sections shared(x)
{
```

```
  #pragma omp section
  { // Al inicio (x = 0 || x = 2)
```

```
    #pragma omp atomic
```

```
    x++;
```

```
  } // Al final (x = 1 || x = 3)
```

```
  #pragma omp section
```

```
  { // Al inicio (x = 0 || x = 1)
```

```
    #pragma omp atomic
```

```
    x += 2;
```

```
  } // Al final (x = 2 || x = 3)
```

```
}
```

```
// Como afirmación debilitada podemos afirmar que x = 3;
```



Tema 4. Concurrency and Synchronization

Productor

● Invariantes globales

```
int buf, p = 0, c = 0; int B[n];
int A[n] = {...}; // Array inicializado
// Invariante Global I: (c <= p <= c+1) &&
//      ( if (p == c+1) then (buf == A[p-1]))
#pragma omp parallel sections
    shared (buf, p , c) private(A,B){
#pragma omp section {
// Invariante local L1: (I) && (p <= n)
while (p < n) {
    // (I) && (p < n)
    ATOMIC await until (p == c);
    // (I) && (p < n) && (p == c)
    buf = A[p];
    // (I) && (p<n) && (p==c) && (buf==A[p])
    p++;
    // (L1) = (I) && (p <= n)
}
} // (L1) && (p == n)
```

Consumidor

```
#pragma omp section {
// Invariante local L2:
// (I) && (c <= n) && (B[0..c-1] == A[0..c-1])
while (c < n) {
    // (I) && (c < n)
    ATOMIC await until (p > c);
    // (I) && (c < n) && (p > c)
    B[c] = buf;
    // (I) && (c < n) && (p > c) && (B[c] = A[c])
    c++;
    // (L2) = (I) && (c<=n) && (B[0..c-1] == A[0..c-1])
}
} // (L2) && (c == n)
} // (I)
```



Tema 4. Concurrencia y Sincronización

• Sincronización

- Bajo el epígrafe "sincronización" se encuentran muchos tipos de sentencias que consiguen que bloques de sentencias de una tarea aparezcan como unidades indivisibles.
- Estas acciones atómicas de grano grueso llevan a usar la sincronización de dos maneras:
 - Exclusión mutua
 - Impedir que dos o más tareas ejecuten simultáneamente un determinado bloque de código con sentencias críticas.
 - Sincronización de condiciones
 - Impedir que una tarea comience a ejecutar un bloque de código antes de que se cumpla una determinada condición.



Tema 4. Concurrencia y Sincronización

Tarea Concurrente ↓

- Propiedades de Seguridad y Viveza (*liveness*)
 - Las propiedades deben ser atributos ciertos en cada posible estado de una tarea.
 - Las tareas concurrentes deben cumplir dos propiedades:
 - **Seguridad**: determina que en ningún momento la ejecución es interrumpida anormalmente y que es capaz de finalizar proporcionando un resultado correcto.
 - **Viveza** (en inglés, *liveness*): determina que la tarea debe ser capaz de progresar en algún momento de la ejecución, por ej. entrar en una sección crítica, que una interrupción sea atendida en algún momento, etc. (Progresar → no implica rapidez)
 - En resumen, el principio de **seguridad** afirma que "**nada malo**" ocurre durante toda la ejecución de una tarea concurrente; mientras que el principio de **viveza** afirma que "**algo bueno**" ocurre en algún **momento de la ejecución**.
- sitienes una sección crítica debe poder entrar



Tema 4. Concurrencia y Sincronización

Seguridad en las tareas concurrentes:

no intersección
crash

- **Garantía de exclusión mutua.** Sección crítica → solo sea ejecutado por 1 tarea
 - El programa concurrente debe garantizar que un trozo de código crítico va a ser ejecutado por una única tarea concurrente cada vez.
- **Ausencia de *deadlock* (interbloqueos).** → No es seguro no acaba nunca entonces
 - El programa concurrente debe garantizar que no todas las tareas van a estar esperando infinitamente a que se produzca una determinada condición. no hay resultado
- **Terminación normal.**
 - El programa concurrente debe garantizar que el programa finaliza o vuelve a un estado de espera análogo al inicial (según el caso), proporcionando una respuesta correcta en todo momento.
- **Reanudación análoga.**
 - El programa concurrente debe garantizar que al finalizar su ejecución, el programa puede volver a ser invocado manteniendo un comportamiento análogo en función del estado externo. Misma acción

No es misma ejecución
ya que el estado o variables han cambiado

limpiar variables basura, memoria

Si este el dato se cumple el deadline
caché



Tema 4. Concurrencia y Sincronización

misma probabilidad para
ejecutar, independencia,
paralelismo
total

• Viveza (*liveness*) en las tareas concurrentes:

• Política de planificación con equidad incondicional:

- Cuando todas las acciones atómicas elegibles son ejecutadas al menos eventualmente.

• Política de planificación con equidad débil:

- En el caso de acciones atómicas condicionales (dependientes de una determinada condición), si cumple la equidad incondicional y se ejecuta cuando su condición pasa a ser verdadera y se mantiene como verdadera al menos hasta que es chequeada por el proceso que ejecuta la acción atómica condicional. (mantener el valor por + tpo, cuidado con cambios rápidos.)

• Política de planificación con equidad fuerte:

- Si tiene equidad incondicional y si la condición pasa a ser cierta un número infinito de veces, aunque cambie el valor de su condición.

no incondicional

Débil: condición dice cual coger (IF) sino se cumple se coge una u otra
si esa condición puede cambiar, solo la compruebo cuando evaluo (Polling)
no lo veo.

Fuerte: misma condición, si se activa se va entrar si o si



Tema 4. Concurrencia y Sincronización

1.Programación con memoria compartida



2.Secciones Críticas: Garantía de Exclusión Mutua

1.Problemas de la modificación de variables compartidas: condiciones de carrera

2.Implementaciones de accesos a Secciones críticas

3.Cerrosjos, Barreras y Semáforos

4.Monitores

5.Programación por Paso de Mensajes

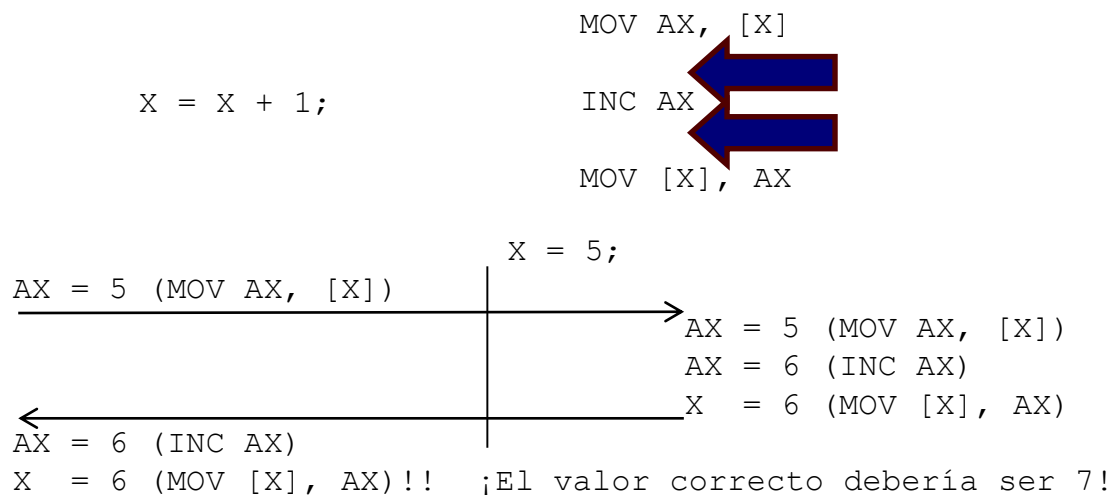


Tema 4. Concurrencia y Sincronización

Secciones Críticas: Garantía de Exclusión Mutua

- Problemas de la modificación de variables compartidas: condiciones de carrera

- Cuando dos tareas concurrentes tratan de acceder a una variable compartida, compiten en el acceso según su velocidad de ejecución => Condición de carrera.



Tema 4. Concurrencia y Sincronización

- Reglas necesarias de las secciones críticas (formal):
 - Exclusión mutua
 - Solo una tarea debe encontrarse dentro de su sección crítica.
 - Ausencia de *Deadlock* / *Livelock*
 - Si dos o más tareas están intentando entrar en su sección crítica libre, una de ellas conseguirá el acceso. Además, ninguna tarea podrá bloquearse dentro de su sección crítica.
 - Ausencia de retardos innecesarios
 - Si una tarea solicita acceder a la sección crítica libre, dicha tarea accederá a la sección crítica en el menor número de ciclos posible.
 - Entrada eventual
 - Todos los procesos que deseen entrar en su sección crítica conseguirán acceso a la misma en algún instante de su ejecución.



Tema 4. Concurrencia y Sincronización

- Reglas necesarias de las secciones críticas (informal):
 - Dos procesos nunca pueden estar simultáneamente dentro de sus regiones críticas.
 - Ningún proceso que se encuentre fuera de su sección crítica puede bloquear a otros procesos.
 - No se pueden realizar ninguna suposición sobre las velocidades y el número de CPUs, ni sobre el grado de ejecución de ninguna tarea.
 - Ningún proceso deberá tener que esperar indefinidamente para entrar a su sección crítica.



Tema 4. Concurrency and Synchronization

- **Implementaciones de acceso a Secciones Críticas**
 - Busy waiting (espera ocupada)
 - Dentro de este tipo se agrupan todas aquellas implementaciones en las que las tareas que están esperando para entrar a la sección crítica están consumiendo ciclos.
 - Dormir/Despertar
 - Este tipo de implementación incluye todas aquellas aproximaciones en las que las tareas en espera pasan a condición de no ejecutables y no consumen ciclos. El S.O. es el encargado de despertarlas y darles acceso a la sección crítica en cuanto ésta se quede libre.
 - Otras implementaciones:
 - Semáforos, barreras, monitores, etc.



Tema 4. Concurrency and Synchronization

• Secciones Críticas con Busy Waiting

• Inhabilitación de interrupciones

- Mediante este mecanismo se impide la conmutación de tareas.
- Es una solución excesivamente drástica ya que impide que ninguna otra tarea pueda actuar con las interrupciones, pudiéndose quedar bloqueada.
- No es útil en sistemas multiprocesador, ya que solo impide el acceso a las tareas en ejecución en el mismo procesador.
- Se utiliza exclusivamente dentro de las tareas que componen el Sistema Operativo.

```
asm {CLI}
```

```
// SECCIÓN CRÍTICA
```

```
asm {STI}
```

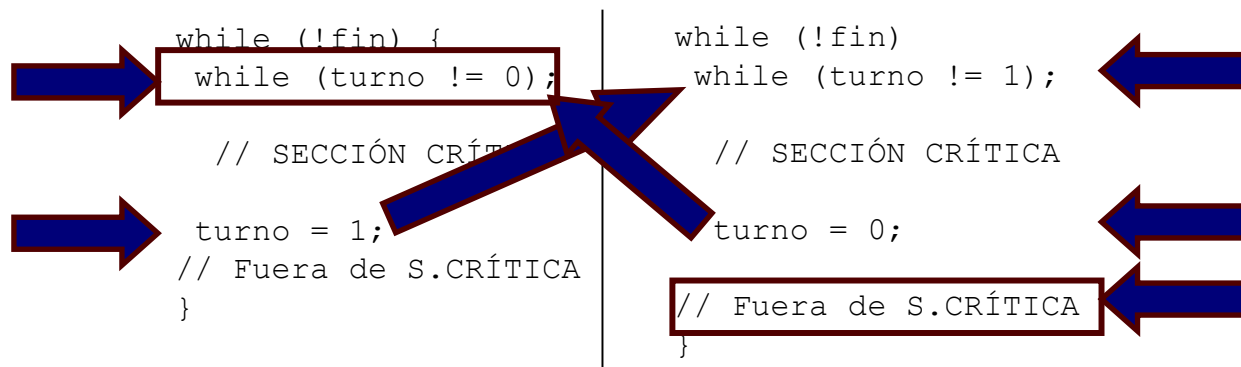


Tema 4. Concurrency and Synchronization

• Secciones Críticas con Busy Waiting

• Alternancia estricta

- Es una solución de grano grueso que emplea 1 variable.
- No es buena solución porque aunque garantiza exclusión mutua, el resto de reglas no se cumplen: una tarea muy lenta podría bloquear a otra muy rápida fuera de sección crítica.



Tema 4. Concurrency y Sincronización

• Secciones Críticas con Busy Waiting

• Spin locks / Cerrojos

- Es una solución de grano grueso que emplea 2 variables (para generalizarlo a **n** procesos, serán necesarias **n** variables).
- El hardware debe implementar una instrucción atómica que sea capaz de comprobar el estado de una variable y realizar la modificación de la misma.

```
ATOMIC <check (!lock) lock = true;>
```

```
// SECCIÓN CRÍTICA
```

```
lock = false;
```



Tema 4. Concurrencia y Sincronización

• Secciones Críticas con Busy Waiting

• Instrucciones Test and Set Lock (TSL)

- Es una solución de grano fino. Implementa en hardware de manera atómica la lectura de una posición de memoria, colocarlo en un registro y almacenar un valor distinto de cero en la posición de memoria original.

```
enter_region:
    tsl register, lock
    cmp register, 0
    jne enter_region
    ret
```

```
leave_region:
    mov lock, 0
    ret
```



Tema 4. Concurrency and Synchronization

• Secciones Críticas con Busy Waiting

• Instrucciones Fetch and Add (FA)

- Es una solución de grano fino. Implementa en hardware de manera atómica la lectura de una posición de memoria, incrementar el contenido de dicha posición y almacenar dicha posición de memoria original, devolviendo el valor antes de incrementar en una variable local.

```
FA [var], incr:
    ATOMIC <tmp = var; var = var + incr; return (tmp);>
```



Tema 4. Concurrency y Sincronización

• Secciones Críticas con Dormir/Despertar

• Primitivas Sleep-Wakeup

- Es una solución de grano grueso.
- **Sleep**: Llamada al sistema que provoca la **suspensión de la tarea** que lo invoca hasta que otro lo despierte.
- **Wakeup**: Llamada al sistema que **despierta al proceso** que se indica.
- Pueden existir problemas de pérdidas de llamadas *wakeup* en los casos en los que las tareas cambien de contexto entre que se chequee una variable y se decida que se ha de realizar una llamada *sleep*.

```
int count = 0;
void producer(void) {
    while (!fin) {
        produce_item ();
        if (count == N)
            sleep();
        enter_item ();
        count++;
        if (count == 1)
            wakeup(consumer);
    }
}
```

```
void consumer (void){
    while (!fin) {
        if (count == 0)
            sleep();
        count--;
        if (count == N-1)
            wakeup(producer);
        consume_item();
    }
}
```

Tema 4. Concurrencia y Sincronización

- 1.Programación con memoria compartida
- 2.Secciones Críticas: Garantía de Exclusión Mutua
- ➔ 3.Cerrojos, Barreras y Semáforos
 - 1.Cerrojos. Barreras
 - 2.Semáforos binarios, semáforos multivaluados
- 4.Monitores
- 5.Programación por Paso de Mensajes



Tema 4. Concurrency and Synchronization

• Cerrojos y Barreras

- Los cerrojos se han mostrado con anterioridad y es el mecanismo más simple para controlar el acceso a las secciones críticas.
 - No tienen control de tareas bloqueadas esperando a que se libere el cerrojo => No permiten políticas de prioridad de tareas.
 - No permiten que puedan entrar más de una tarea a una sección protegida de código, aunque ésta lo admita => Dificultad para sincronizar más de 2 tareas a la vez.
- Las barreras permiten sincronizar muchas tareas a la vez: Bloquean todas las tareas hasta que se encuentren esperando n tareas. Para todas las tareas hasta que se cumpla una condición
 - Permiten sincronizar diferentes etapas de un algoritmo.



Tema 4. Concurrency y Sincronización

Este mecanismo solo funciona una vez, hay que reiniciarlo -> y tiene que hacerlo el primero que pase o el último

● Implementaciones de las barreras

● Contadores compartidos

```
loop:      FA (count, 1);
          if (count < n) {
              FA(count, -1);
              goto loop;
          };
```

- Pueden presentar problemas de reanudación análoga tras su uso (hay que realizar una puesta a cero, en sección crítica).

- Para evitarlo se puede asumir que el último que sea capaz de “pasar sin espera”, realice de manera atómica una puesta a cero => Complejidad y retardo innecesario.

● Mediante banderas y coordinadores

- En lugar de incrementar un contador, cada vez que una tarea llegue a una barrera, activa una bandera en un vector de banderas compartidas.

- Existe una tarea especial llamada “coordinador” que se encarga de ir chequeando sistemáticamente dicho vector de banderas y mientras que haya alguna bandera inactiva, no permite continuar al resto de tareas.

- Este método permite el control de prioridad e incluso es capaz de reanudar en caso de bloqueo prolongado de alguna tarea.



Tema 4. Concurrencia y Sincronización

• Semáforos binarios

- En 1965 E.W. Dijkstra propuso la utilización de una abstracción nueva que permitiera la sincronización y el control de acceso a secciones críticas.
- Los semáforos binarios están compuestos por una variable booleana y dos instrucciones atómicas:
 - UP(semáforo): pone el valor del semáforo a uno.
 - DOWN(semáforo): pone el valor del semáforo a cero. En caso de que el semáforo ya tuviese el valor 0, la tarea se bloquea hasta que en algún momento el semáforo tome valor a uno.
- La reactivación de las tareas bloqueadas depende del sistema, que será el encargado de escoger una de las tareas bloqueadas y reactivarla.



Tema 4. Concurrency y Sincronización

• Semáforos multivaluados

- Una generalización de los semáforos binarios son los semáforos multivaluados. Estos utilizan una variable entera no negativa y dos instrucciones atómicas:
 - UP(semáforo): Incrementa en uno el valor del semáforo.
 - DOWN(semáforo): Decrementa en uno el valor del semáforo. Si antes de incrementar el valor del semáforo es cero, la tarea se bloquea.
- Mediante esta generalización se puede permitir el paso a más de una tarea. Mediante la inicialización del semáforo a un valor entero superior a cero se controla cuántas tareas pueden solicitar la entrada.
- Suelen estar implementados como llamadas al sistema con inhibición de interrupciones y/o utilizando la instrucción en ensamblador TSL (para los semáforos binarios) o la instrucción FA (para los semáforos multivaluados), además de utilizar el mecanismo sleep/wakeup de manera protegida dentro del Sistema Operativo.



Tema 4. Concurrencia y Sincronización

- 1.Programación con memoria compartida
- 2.Secciones Críticas: Garantía de Exclusión Mutua
- 3.Cerrojos, Barreras y Semáforos
- ➔ 4.Monitores
 - 1.Exclusión mutua basada en Monitores
- 5.Programación por Paso de Mensajes



Tema 4. Concurrencia y Sincronización

• Monitores

- Esta primitiva de sincronización fue propuesta por Hoare en 1974. Los monitores son un conjunto de procedimientos, variables y estructuras de datos agrupados en forma de clase o módulo especial dentro de los lenguajes orientados a objetos.
- Todas las variables están definidas como privadas y no son accesibles desde fuera de los monitores.
- Los monitores solo permiten que una tarea esté ejecutándose dentro de ella. Cuando una tarea intenta acceder a un monitor, lo primero que hace es comprobar si hay alguna otra tarea dentro, en cuyo caso se suspende hasta que el otro proceso salga.
- Suelen tener dos funciones miembros que controlan las variables de condición de los monitores:
 - WAIT(condicion): La tarea se bloquea en la lista de espera asociada a la variable *condicion*. En ese momento libera el monitor para que otra tarea pueda entrar al monitor.
 - SIGNAL(condicion): El sistema despierta a una de las tareas que se encuentre bloqueada en la lista de la variable *condicion*.
- Conceptualmente son similares a SLEEP/WAKEUP.



Tema 4. Concurrency y Sincronización

• Ejemplo de monitor para acceso a un buffer:

```
monitor Buffer {

    char buf[n];
    int  front = 0, rear = 0, count = 0;
    cond not_full, not_empty;

    void putElement (char c) {
        while (count == n) wait (not_full);
        buf[rear] = c; rear = (rear + 1) % n; count++;
        signal (not_empty);
    }
    char getElement (void) {
        char element;
        while (count == 0) wait (not_empty);
        element = buf[front]; front = (front + 1) % n; count--;
        signal (not_full);
        return (element);
    }
}
```



Tema 4. Concurrency and Synchronization

1. Programación con memoria compartida
2. Secciones Críticas: Garantía de Exclusión Mutua
3. Cerrojos, Barreras y Semáforos
4. Monitores

→ 5. Programación por Paso de Mensajes

1. Conceptos de los programas distribuidos: envío y recepción de mensajes
2. Paso de mensaje síncrono (bloqueante) vs. Paso de mensaje asíncrono (no bloqueante)
3. Llamadas a procedimientos remotos (RPC)
4. Rendezvous



Tema 4. Concurrencia y Sincronización

• Conceptos de los programas distribuidos: envío y recepción de mensajes

- En la **programación distribuida** las tareas no tienen acceso a **variables compartidas**: cada tarea tiene su propia memoria local totalmente independiente del resto.
- La **sincronización** se realiza mediante comunicación **explícita** entre tareas: unas tareas envían datos a otras tareas, que deben estar **esperando dichos envíos**.
- Las **recepciones** pueden ser **identificadas**, provenientes de otra **tarea específica**, o bien generales, provenientes de cualquier otra tarea. Puede escuchar lo que se le indique, sea general (se escucha a todos, puede venir de cualquier nodo o tarea) o específica
- Es la **forma más natural de programación** ya que **imita** el **comportamiento humano** de paso de turno.



Tema 4. Concurrency y Sincronización

• Paso de mensaje síncrono (bloqueante) vs. Paso de mensaje asíncrono (no bloqueante)

- La sincronización se realiza debido a que las instrucciones de envío/recepción son bloqueantes.

- Hay tres tipos de envíos:

- **Asíncronas o No Bloqueantes:** El emisor continúa el proceso independientemente de si el receptor ha recibido o no el mensaje. (Estándar POSIX)

- **Síncronas o Bloqueantes:** El emisor continúa procesando solo cuando el mensaje ha sido recibido por el receptor.

- **Invocación remota:** El emisor continúa procesando cuando recibe un mensaje de confirmación de recepción por parte del receptor (Implementado en ADA)

- Todas las recepciones son síncronas ya que una tarea no puede procesar un mensaje que no ha sido recibido. La tarea que espera recibir se bloquea hasta que recibe un cierto dato.

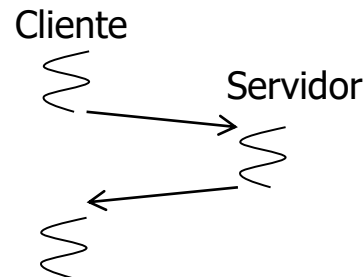
- Utilizando las **recepciones síncronas** generales se puede controlar el acceso a las **secciones críticas de código** o implementar **barreras de sincronización**.



Tema 4. Concurrency y Sincronización

• Llamadas a procedimientos remotos (RPC)

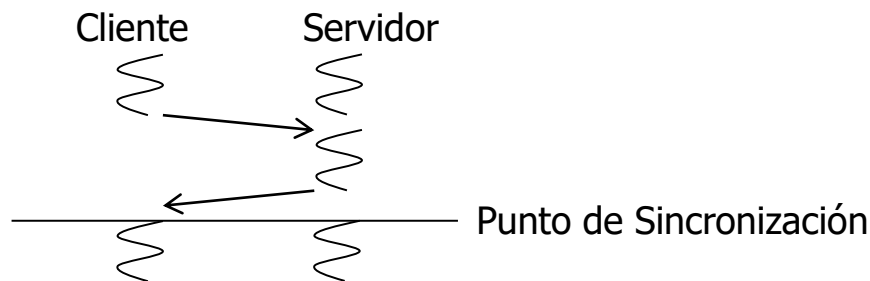
- Es un mecanismo de ejecución de procedimientos basado en el **protocolo cliente-servidor**.
- En el mecanismo cliente-servidor: el servidor exporta una serie de funciones y los clientes solicitan la ejecución de dicha función, bloqueando mientras tanto la ejecución del código de la tarea cliente.
- Cuando se realiza una llamada RPC a un servidor, se crea un hilo que atiende dicha petición, ejecuta la función asociada, envía la respuesta al cliente y finaliza dicho hilo, liberando todos los recursos utilizados.



Tema 4. Concurrency and Synchronization

• Rendezvous

- El mecanismo RPC proporciona comunicación y exclusión mutua, pero no proporciona de manera directa sincronización.
- El mecanismo de Rendezvous (en francés, cita o punto de encuentro) permite además de la comunicación entre tareas, la sincronización entre las mismas.
- Tras la ejecución de un Rendezvous la tarea llamadora y la tarea receptora continúan su ejecución a la vez de manera síncrona.



Preguntas ...

Volver

